

NETRA

City-Wide AI Engine for Multi-Camera ANPR Trajectory Tracking and Urban Traffic Analytics

PROBLEM STATEMENT ID	THEME	PS CATEGORY
127	<confirm on portal>	Software
TEAM ID	TEAM NAME	INSTITUTE
<fill in>	<fill in>	<fill in>

A working prototype, not a concept. Multiple camera feeds are processed in parallel, number plates are read and agreed across many frames, vehicles are matched between cameras with a confidence score, and their journeys through the network are rebuilt and shown on a live dashboard.

Proposed Solution

NETRA — a software layer that runs on top of the traffic cameras a city already has.

What the system does

Our system takes video from several cameras at once, finds vehicles, reads number plates, and then works out which sightings belong to the same vehicle. From that it reconstructs where each vehicle went and what the traffic is doing.

- **Detects and tracks** vehicles in each camera, and keeps one ID per vehicle while it is in view.
- **Reads the plate many times** — 40 to 90 frames per vehicle — and votes on the answer letter by letter instead of trusting a single frame.
- **Matches vehicles across cameras** and gives every match a confidence score.
- **Rebuilds the journey**: route, distance, time per leg, speed against the corridor's usual pace.
- **Produces analytics** — congestion, origin–destination flow, camera performance, alerts.

How it addresses the problem

- Today each camera is an island. You can count what passes a junction, but you cannot follow one vehicle across the city.
- We connect those separate sightings, so "where did this vehicle come from, and how long did it take" becomes a question the system can answer.
- No new hardware. It works with cameras that are already installed.

What actually happens to one vehicle



The highlighted steps are where most of our own work sits. Detection and tracking we use off the shelf — deciding *which readings belong to the same vehicle* is the hard part.

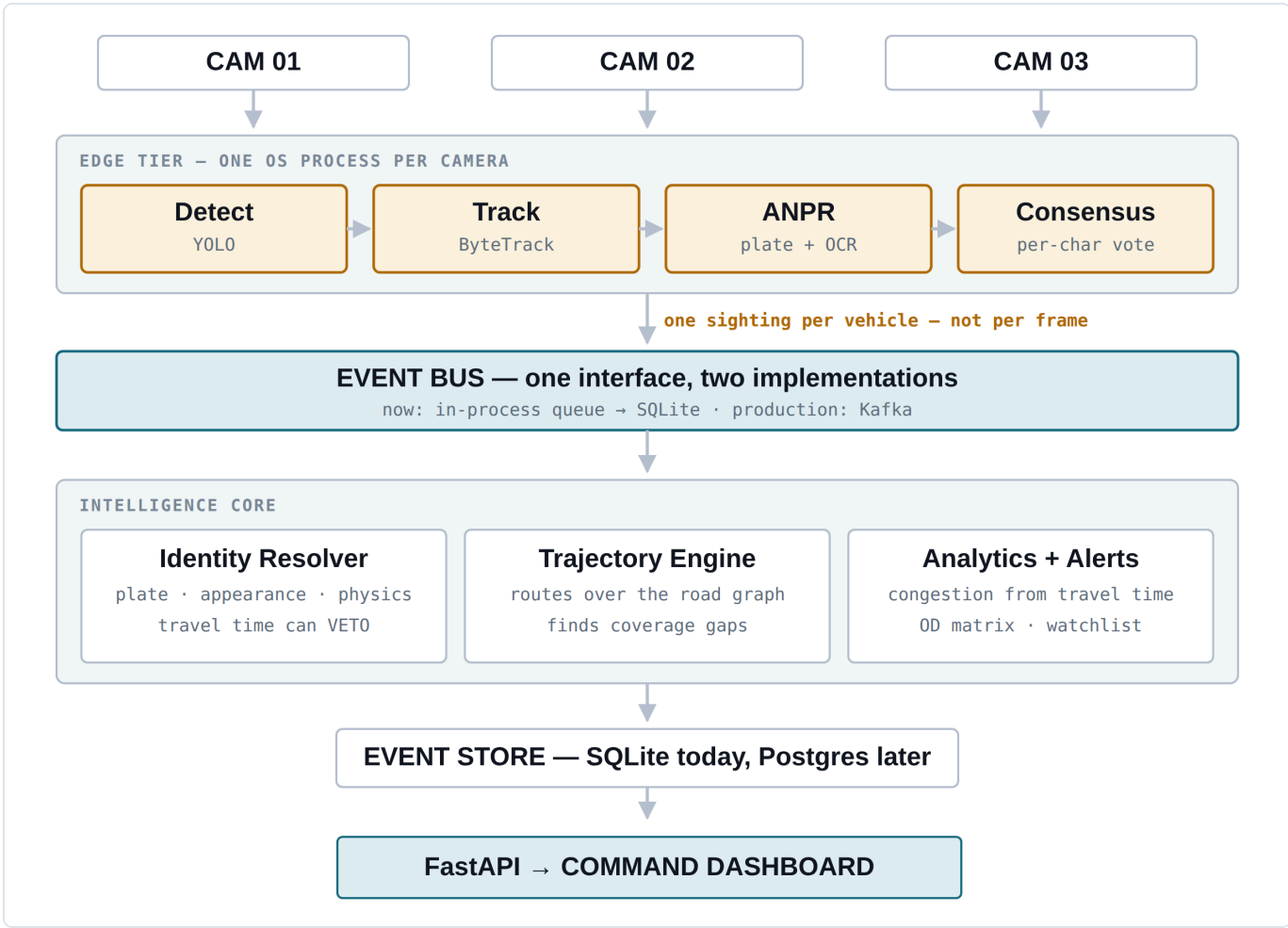
What makes it different

We treat a plate read as a guess, not a fact. Every plate, every match and every journey carries a confidence, and the system says when it is unsure.

- **Three signals, one of which can veto.** A match is scored on plate similarity, appearance, and whether the journey is physically possible. If the camera positions and the timestamps imply 3,000 km/h, we reject the match — even when the plates are identical.
- **We show what we refused.** Rejected matches are stored with the reason. It is easy to show a system that links things; showing what it would *not* link is the honest proof it is reasoning.
- **Plate grammar for Indian plates.** We know the format, so a "0" where only a letter can sit is corrected to "O". We only repair towards plates that could actually exist.
- **Congestion from travel time, not counts.** Forty vehicles moving freely is not congestion. Four crawling is.
- **Coverage gaps.** If a vehicle certainly passed a camera that recorded nothing, we report it — that is a camera fault a city can act on.

Technical Approach

Everything below is open source and free to use. No paid APIs, no licence keys.



Technologies we use

Detection	YOLO (Ultralytics), pretrained on COCO
Tracking	ByteTrack
ANPR	fast-alpr — plate detector + OCR, ONNX Runtime
Backend	Python, OpenCV, FastAPI, WebSocket
Storage	SQLAlchemy on SQLite; same models run on PostgreSQL
Frontend	React, Vite, Leaflet, Recharts
Maps	OpenStreetMap via CARTO tiles, cached locally

How it runs

- One operating-system process per camera. Python's GIL means threads would queue up behind each other, so separate processes actually run in parallel.
- Each worker publishes to an **event bus**. Right now that is an in-process queue writing to SQLite. Swapping it for Kafka is a second implementation of the same two methods, not a rewrite.
- Consensus happens inside the worker, so a camera writes **one record per vehicle** rather than one per frame. That is roughly 50× fewer writes.
- The reason we chose this split is that it is also where the network goes in a real deployment — the same worker code runs on an edge box beside the camera.

Feasibility and Viability

The prototype is built and running. These are the things that can still go wrong, and what we did about them.

Built

Full pipeline working end to end on recorded footage, from video in to dashboard out.

One laptop

Three cameras processed in parallel. Runs on CPU; faster on Apple Silicon or a GPU.

₹0

Every model, library and map tile is free. No accounts, no keys. Runs offline.

262 tests

Logic tests for tracking, plate consensus, matching, journeys and analytics. Run in about five seconds.

Risks, and what we do about them

RISK	WHY IT MATTERS	HOW WE HANDLE IT
Plates too small to read	Municipal cameras are mounted for flow, not identification. Compression makes it worse.	We skip anything under a minimum plate width instead of guessing, and report the confidence. For the demo we record our own footage at angles where plates are actually legible.
OCR misreads characters	One wrong character makes two sightings of the same car look like different vehicles.	Per-character voting across every frame, repair against the Indian plate format, and matching that knows which characters get confused (0/O, D/O, 1/I).
Wrong cross-camera matches	A false match invents a journey that never happened. Worse than missing one.	The travel-time check can veto any match. And we never claim two vehicles are the same on appearance alone — two silver hatchbacks are not the same car.
Scaling past a few cameras	A laptop will not run a whole city.	Process per camera, event bus, and a database schema that already runs on PostgreSQL. The migration is a config change, not a redesign.
Something breaking live	A demo that fails looks like a system that does not work.	A preflight command checks twenty things in ten seconds, the database is pre-seeded so nothing opens empty, and map tiles are cached so the venue's wifi does not matter.

Impact and Benefits

Who uses it, and what changes for them.

Who it is for

- **Traffic police** — trace a vehicle of interest across the network in seconds, with the confidence of every step shown rather than hidden.
- **City transport departments** — see which corridors are actually slow, measured from real journey times.
- **Urban planners** — origin-to-destination patterns across the whole network, which is the input to signal timing and road investment decisions.
- **Camera operations teams** — a list of cameras that vehicles definitely passed but which recorded nothing.

What it changes

- A question that currently takes hours of manually scrubbing footage becomes a search box.
- Congestion stops being a guess from vehicle counts and becomes a measured number.
- Camera faults get found by the system instead of by someone noticing months later.

A concrete example

A complaint comes in about a vehicle seen around 10:40. Today someone pulls footage from that junction, finds it, then has to guess which camera to check next and scrub through that one too. It takes hours and often fails. With our system you type the plate and get the route back: **CAM01 10:42:13 → CAM04 10:46:51 → CAM07 10:52:09** — 9.8 km, 12 minutes, with a confidence on every step and the raw plate readings behind it if anyone wants to check.

Benefits

Social	Faster response to incidents and vehicle-related complaints. Better emergency routing when you know which corridors are moving.
Economic	Uses cameras that are already installed and paid for. The whole stack is open source, so there are no per-camera licence fees — which is usually the reason city-scale ANPR does not get deployed.
Environmental	Better signal timing from real origin–destination data means less idling, which means less fuel burnt and less local air pollution.
Operational	The system reports its own blind spots, so the camera network gets maintained on evidence rather than on complaints.

What we are careful about

This is vehicle data, and we are aware of what that means. The system never asserts an identity it cannot justify, every match is auditable, and a watchlist hit is flagged for a human to confirm rather than treated as fact. A real deployment would also need retention limits and access controls, which our prototype does not implement.

Research and References

What we read and what we built on.

Models and libraries

Ultralytics YOLO	Vehicle detection github.com/ultralytics/ultralytics
ByteTrack	Multi-object tracking by associating every detection box arxiv.org/abs/2110.06864
fast-alpr	Plate detection and OCR, ONNX, MIT licence github.com/ankandrew/fast-alpr
FastAPI	API and WebSocket layer fastapi.tiangolo.com
Leaflet	Map rendering leafletjs.com
OpenStreetMap / CARTO	Basemap tiles, free, no key carto.com/basemaps

How we picked these

We did not want to choose an OCR model because it was popular, so we wrote a small script that runs each candidate over the same set of plate crops and prints character accuracy and speed. The same idea applies to the matching engine: we score it against a written record of which vehicle actually passed which camera, rather than assuming it works. Where we have not measured something yet, our results file says "not yet measured" instead of estimating.

Background reading

CityFlow	The standard benchmark for multi-camera vehicle tracking and re-identification. We used it to understand how this problem is normally evaluated. arxiv.org/abs/1903.09254
AI City Challenge	Annual work on city-scale multi-camera tracking. Useful for seeing where the hard parts actually are. aicitychallenge.org
Roboflow Universe	Indian number plate datasets, for fine-tuning the plate detector. universe.roboflow.com

Our own work

The prototype is written and running — around 70 source files across the vision pipeline, matching engine, API and dashboard, with 262 automated tests. We also collect our own multi-camera footage with a written record of which vehicle passed which camera and when, so we can measure how accurate the matching actually is rather than assert that it works.